Report - Version 3: Preprocessing Agent

Introduction

Version 3 extends the Building Agentic AutoML project with automatic preprocessing experiments.

The agent still receives a dataset and target, detects the task and feature types, inspects the dataset, trains a LightGBM baseline, evaluates it, and returns a structured state. The new capability is that it now tries multiple preprocessing strategies, evaluates them under the same validation protocol, and keeps the best result.

This version focuses on the following flow:

```
Dataset + target
-> Detect task and features
-> Inspect dataset
-> Try preprocessing strategies
-> Evaluate each strategy
-> Select best result
-> Return state + experiments
```

The architecture remains intentionally simple: one model family, one train-validation split, two preprocessing strategies, and no optimization. Version 3 adds experimentation over preprocessing while preserving the previous pipeline.

Goal of Version 3

The goal of Version 3 is to make the agent act on data preparation choices instead of using a single fixed preparation step.

The agent should now be able to:

- load a dataset and target and detect the task;
- identify numerical and categorical features;
- inspect dataset dimensions, target behavior, data types, missing values, and cardinality;
- try a small set of preprocessing strategies;
- learn preprocessing parameters only from the training data;
- evaluate the strategies with the same model and validation split;
- select the best preprocessing result and store the experiments in the final state.

Adult Income remains the main classification example so that the only new concept introduced in this version is preprocessing.

Architecture

The flow of Version 3 is:

```
Dataset + Target -> Task/Features -> Data Inspection -> Preparation
-> Preprocessing Experiments -> Model/Metric -> Evaluation -> Best Result -> State
```

The function `agent` still coordinates the complete flow directly. The new preprocessing experiment loop is added without introducing new model families, cross-validation, hyperparameter search, or a planner.

Task Detection

Task detection is unchanged from Version 2. The agent uses the number of unique target values as a simple rule: at most 20 unique values means classification; otherwise the task is treated as regression.

```
def detect_task(df, target):
    n_unique = df[target].nunique()
    if n_unique <= 20:
        return "classification"
    return "regression"
```

For the Adult Income target `income`, the result is `"classification"`.

Feature Detection

Feature detection is also unchanged. For Adult Income, the agent detects 6 numerical features and 8 categorical features.

```
numerical = ['age', 'fnlwgt', 'education_num',
             'capital_gain', 'capital_loss', 'hours_per_week']

categorical = ['workclass', 'education', 'marital_status', 'occupation',
               'relationship', 'race', 'sex', 'native_country']
```

Data Inspection

The Data Inspector introduced in Version 2 is preserved unchanged. For Adult Income it reports 48,842 rows and 15 columns, two target classes, feature dtypes, missing values, and cardinality.

The main missing values remain `workclass = 2799`, `occupation = 2809`, and `native_country = 857`. The inspection remains descriptive; it is stored in the state but does not decide which preprocessing strategies are available.

Preprocessing

This is the new capability introduced in Version 3. The agent compares two simple strategies:

```
native -> keep missing values and let LightGBM handle them
impute -> median for numerical features, mode for categorical features
```

The train-validation split is created before preprocessing. Median values, modes, and categorical vocabularies are therefore learned only from the training data, avoiding validation leakage.

Categorical columns are converted using a `pandas.CategoricalDtype` built from categories observed in the training set, so training and validation use the same categorical vocabulary.

Baseline Model and Metric Selection

Model and metric selection remain unchanged. Classification uses LGBMClassifier with ROC AUC, while regression uses LGBMRegressor with RMSE. Both models use `random_state=42`.

```
classification -> LGBMClassifier + ROC AUC
regression     -> LGBMRegressor + RMSE
```

Training and Evaluation

Each preprocessing strategy is evaluated with the same model family and the same train-validation split. The split uses `test_size=0.2` and `random_state=42`; classification also uses stratification.

On Adult Income, the notebook obtains:

```
native -> ROC AUC = 0.9311658417981501
impute -> ROC AUC = 0.930687503244851
best_preprocessing = 'native'
```

Because ROC AUC is maximized, `native` is selected. The result also shows that preprocessing is treated as an experiment: imputation is not assumed to be better automatically.

Agent

In Version 3, the Agent performs the following operations:

```
1. load dataset and target
2. detect task and feature types
3. inspect the dataset
4. prepare X and y
5. select the metric
6. try, train, and evaluate each preprocessing strategy
7. select the best experiment
8. return the final state
```

A simplified state now contains:

```
{
    "task": "classification",
    "model": "LGBMClassifier",
    "metric": "roc_auc",
    "experiments": [
        {"preprocessing": "native", "score": 0.9311658417981501},
        {"preprocessing": "impute", "score": 0.930687503244851}
    ],
    "best_preprocessing": "native",
    "score": 0.9311658417981501
}
```

The state therefore records both the final decision and the evidence used to compare the available preprocessing strategies.

Regression Test

The same agent is tested on `simple_regression.csv`. It detects regression automatically, uses RMSE, and evaluates the same two preprocessing strategies.

```
native -> RMSE = 1.8760451113860066
impute -> RMSE = 1.8776921187920332
best_preprocessing = 'native'
```

Because RMSE is minimized, `native` is again selected. This confirms that the experiment-selection logic adapts correctly to both metric directions.

What Version 3 Teaches

Version 3 introduces a fundamental AutoML idea: data preparation choices should be evaluated empirically rather than applied as fixed assumptions.

The main idea becomes:

```
observe -> inspect -> try -> evaluate -> compare -> select -> return
```

The agent now keeps a small experiment history and selects the best preprocessing result. This is the first version where the state records alternative attempts rather than only one final training run.

The evolutionary pattern remains explicit: Version 2 learned to observe the dataset; Version 3 adds a minimal action loop over preprocessing while keeping model selection and validation strategy unchanged.

Limitations

Version 3 still has intentional limitations:

- task detection still relies only on the number of unique target values;
- only two preprocessing strategies are compared: native handling and simple imputation;
- numerical imputation uses only the median and categorical imputation only the most frequent value;
- the Data Inspector does not yet determine which preprocessing strategies should be tried;
- no scaling, encoding alternatives, outlier treatment, or feature engineering are explored;
- only one model family, LightGBM, is used;
- evaluation still uses a single train-validation split and no cross-validation;
- there is no hyperparameter optimization, planner, or multi-agent architecture yet.

These limitations are deliberate. The purpose of Version 3 is preprocessing experimentation, not model selection or optimization.

Conclusion

Version 3 extends Building Agentic AutoML with a small preprocessing experiment loop while preserving the simplicity of the previous versions.

The agent can now compare native missing-value handling with simple imputation, learn preprocessing parameters only from training data, evaluate both choices under the same protocol, store the experiments, and return the best strategy.

The key lesson is that an AutoML agent should not only inspect data but also test alternative preparation choices and retain the evidence behind its decision. This prepares the project for Version 4, where the next new concept can be model selection.